AI Image Generator

4. Workflows

Version 1.0
22 October 2026

Project owner: Frazile AI
Platform: NVIDIA GeForce RTX 3090, Windows 11, Python 3.12

Stable Diffusion XL + Real-ESRGAN, served locally with Gradio, with per-image GPU telemetry recorded in SQLite.

4. Workflows

4.1 First-time setup

1. Confirm the GPU and driver are visible: run `nvidia-smi` in a terminal. It must list the card and a driver version.
2. Open PowerShell in the project folder.
3. Create and activate a virtual environment:

```
python -m venv .venv
.\.venv\Scripts\Activate.ps1
```

4. Install the CUDA build of PyTorch, then the remaining dependencies:

```
pip install torch torchvision --index-url https://download.pytorch.org/whl/cu126
pip install -r requirements.txt
```

5. Verify CUDA is active:

```
python -c "import torch; print(torch.cuda.is_available(), torch.cuda.get_device_name(0))"
```

6. Confirm the upscaler weights exist at models\RealESRGAN_x4plus.pth. If not, download them from the Real-ESRGAN v0.1.0 release page into that folder.
7. Start the app once. The first start downloads the 6.7 GB SDXL model into the Hugging Face cache and can take several minutes depending on bandwidth.

4.2 Daily use: generating an image

1. Double-click run-app.bat, or run `.\.venv\Scripts\python.exe app.py` from the project folder.
2. Wait about 30 seconds for "Model ready" and "Upscaler ready" in the console. The browser opens http://127.0.0.1:7860 automatically.
3. Type a prompt. As you type, the line under the box shows the interpreted settings, for example "1344x768 | upscale to 3840x2194 | 40 steps | guidance 7".
4. Optionally edit the negative prompt.
5. Press Generate or Enter. A progress bar shows the step count and, if applicable, the upscale stage.
6. The image appears in the gallery. The details panel below shows timing, settings, GPU telemetry and the exact prompt sent to the model.
7. Both files are already saved in outputs\. Nothing further is required.



Figure 4.1: "a lighthouse on a rocky cliff at golden hour, cinematic, 16:9, 4k, seed 42" rendered at 1344x768 and upscaled to 3840x2194.

4.3 Producing 4K and 8K output

1. Include `4k`, `8k`, `2k` or an exact size such as `3840x2160` in the prompt.
2. The parser sets an upscale target and raises steps to 40 for a cleaner base.
3. Generation runs at the native bucket for the aspect ratio (for 16:9 that is 1344x768).
4. The upscaler runs one x4 pass for 4K, or two passes for 8K with an intermediate resize so the second pass lands on the target.
5. If an exact size was given, the result is centre-cropped to it.
6. The gallery and History show the upscaled image; the native image is also kept on disk.

Recommended practice: iterate at native size until the composition is right, then add `4k` and the same `seed N` to the prompt to upscale that exact image.

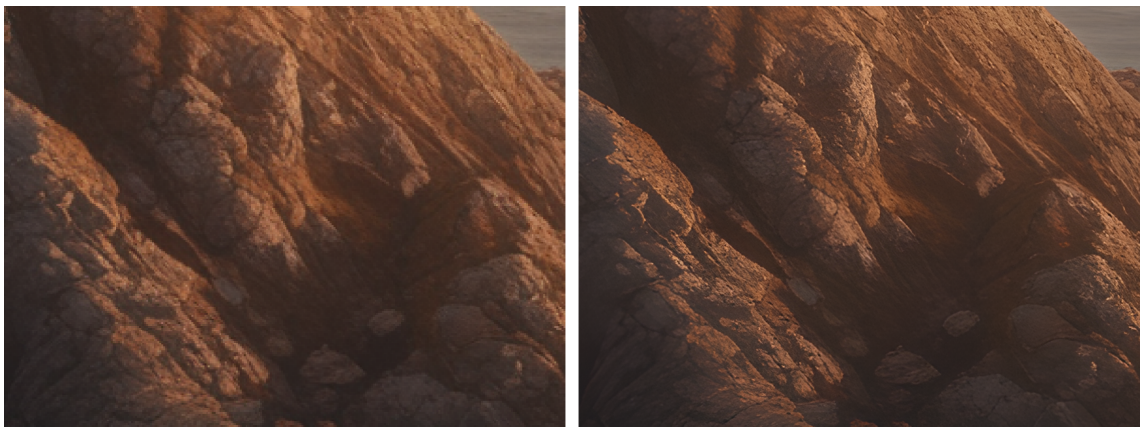


Figure 4.2: The same 300x225 region of the lighthouse cliff. Left: native pixels enlarged. Right: Real-ESRGAN output. The upscaler reconstructs rock texture rather than blurring.

4.4 Reviewing past generations

1. Open the History tab. The top line shows totals: number of images, GPU seconds, average and extreme timings, VRAM, hottest temperature and peak power.
2. Type part of a prompt in the search box and press Enter to filter, or press Refresh to reload.
3. Click a row. The image (upscaled if one exists) and its full details appear below the table.
4. For scripted access, run from the project folder:

```
.\.venv\Scripts\python.exe db.py          # last 20 rows
.\.venv\Scripts\python.exe db.py 12       # every field of row 12
.\.venv\Scripts\python.exe db.py stats    # aggregates
```

4.5 Batch generation from the command line

The original script is retained for scripted use. It loads the model per run, so it is best for batches:

```
.\.venv\Scripts\python.exe generate.py "a red sports car" --steps 40 --seed 42 --count 4
.\.venv\Scripts\python.exe generate.py --help
```

It writes to outputs\ but does not record to the database or upscale.

4.6 Upscaling an existing image

```
.\.venv\Scripts\python.exe upscaler.py outputs\some_image.png 3840
```

The result is written next to the input with an `_up3840` suffix.

4.7 Stopping and restarting

- Close the console window, or press Ctrl+C in it, to stop the server. Loaded models are released.
- Restart with run-app.bat. Startup is about 30 to 45 seconds because the models are reloaded from disk.
- If port 7860 is reported busy, a previous instance is still running; end it from Task Manager (python.exe) or change the port in the last line of app.py.

4.8 Backup

Copy two things: the outputs\ folder and generations.db. The models can be re-downloaded and the virtual environment can be rebuilt from requirements.txt.

4.9 Development workflow

1. Edit a module. The parser and database have self-tests: `python prompt_parser.py` prints how sample prompts are interpreted, and `python db.py stats` exercises the schema.
2. Check syntax quickly with `python -W error -c "import ast; ast.parse(open('app.py', encoding='utf-8').read())"`.
3. Restart the app. For automated checks, gradio_client can call the app's `/generate` and `/refresh_history` endpoints, which is how the increments in this project were verified.